

CNN程式練習

環境與資料準備

- `from tensorflow.keras.datasets import mnist`
- `(X_train, Y_train), (X_test, Y_test) = mnist.load_data()`

匯入模組：使用 TensorFlow 的 Keras 介面

載入數據：自動從雲端下載 60,000 張訓練圖與 10,000 張測試圖。

資料前處理

- `X_train.reshape(..., 28, 28, 1)`
- `X_train /= 255`
- `Y_train = to_categorical(Y_train)`

維度調整：CNN 需要長、寬、通道數。

這裡將 2D 圖片補上一個通道位（灰階=1），符合模型輸入規範。

正規化 (Normalization)：將像素 0~255 縮放至 0~1。

標籤轉換：將數字標籤轉換為向量（如 `[0,0,0,0,0,1,0,0,0,0]`）。

這是為了對齊模型最後一層的機率輸出。

CNN 架構設計

- Conv2D(8, kernel_size=(5, 5), activation="relu")
- MaxPooling2D(pool_size=(2, 2))

卷積層 (Convolution)：利用kernel掃描影像，提取細微的特徵。

池化層 (Max Pooling)：將 2*2 的區域壓縮成 1 個點，取最大值。

全連接層與輸出

- Flatten()
- Dense(64, activation="relu")
- Dense(10, activation="softmax")

Flatten：將 **2D** 的特徵圖拉直成 **1D** 向量，才能進入分類層。

ReLU 激活函數：

在隱藏層提供非線性能力，讓模型學習，能處理複雜圖形。

Softmax 輸出層：輸出 **10** 個機率值，總和為 **1**。

這代表 **AI** 對 **0** 到 **9** 每個數字的「信心程度」。

模型編譯與訓練

- `model.compile(loss="categorical_crossentropy", optimizer="adam")`
- `model.fit(..., validation_split=0.2, epochs=10)`

損失函數 (Loss)：評估預測與真實標籤的差距。

優化器 (Adam)：根據 Loss 自動調整權重。

驗證集 (Validation Split)：在訓練時保留 20% 的資料不參加訓練，僅用來監控模型有沒有「過擬合（Overfitting）」。

實務應用

- `files.upload()`
- `cv2.bitwise_not(img_resized)`
- `np.argmax(prediction)`

上傳：從電腦選檔。

影像處理 (**CV**)：將真實照片轉為與訓練集相同的規格。

提醒：若照片是白底黑字，必須執行反轉顏色，否則 **AI** 會看不懂。

決策 (**Argmax**)：

從 **10** 個機率中找出最高的那個，即為最終辨識出的數字。